

INTERNAL ASSESSMENT

NAME	RATHIN BAGCHI
ROLL NO.:	2414517254
COURSE	BCA
SEMESTER	IV
SUBJECT	PYTHON PROGRAMMING – DCA2205

SET – I

Question 1

a) Data Types in Python

Python supports several data types which help to determine the type and operations that can be performed on the variables. The first type is Numeric Types including integers (int), decimal numbers (float) and numbers with real and imaginary parts (complex). These are employed in mathematical calculations and judgments and in scientific calculations. The second type is String (str), which is a sequence of characters used to represent the text. Python strings are immutable; They do not allow changes to their content once created. They can be used to perform different operations such as slicing, concatenation, and formatting. The third type is Boolean (bool) that could contain only two values: True or False. Typically used in conditional statements and logical operations. The fourth type is List –

ordered and mutable collection storing elements of different data types. Lists can also be modified after they are created, items can be inserted into them, items can be deleted from them or existing items can be removed. A 5th is Tuple which is immutable and like a list. After a tuple is formed, elements of it cannot be changed. The purpose of tuples is to use them if the data should not be changed during a program's execution. The 6th type is Dictionary (dict), which is used to store key-value pairs of data. Keys are unique and are associated to value. Dictionaries do not have any particular order, are mutable, which means that they can be changed, and are best suited for structuring data. The 7th is Set, which is an unordered collection of unique elements. Sets automatically remove duplicate elements and are capable of mathematical operations with sets, such as union and intersection. The final type, NoneType, is the type of object that is missing, and is represented with the help of a Python keyword, None. It is frequently used to give a default value or to say that a variable has not been assigned.

b) Membership and Identity Operators

Membership operators are used for determining if a value is found in a sequence, including sequences of numbers, or any other type of sequence, namely a list, tuple, string, or dictionary. Python comes with two membership operators: in and not in. The in operator will return True if the value appears in the sequence. The not in operator returns true if the value is not contained in the sequence. Rather the operators used in conditional statements to validate information or carry out efficient searches in collections. Identity operators compare the addresses of two values instead of the values themselves.

selves. There are two identity operators in Python, `is` and `is not`. The `is` operator will return true if the two variable reference the same object in memory, not the same value. To determine if two variables refer to different objects in the memory, use the `is not` operator. It is important to differentiate that `==` checks the value of an object, and `is` checks if they are actually the same object (at the same memory location). First of all, two variables can contain the same value, yet they are completely different objects in memory and so `is` evaluates to True but `!=` evaluates to False. The following behavior is particularly important for mutable objects, such as a "list".

Question 2

a) Role of File Modes in File Handling

In python, the file modes allow you to control in what manner the file is opened and what tasks you can perform on the file. The mode argument in built-in `open()` function indicates the purpose of operating the file.

The read mode `'r'` opens up a file for reading only. The file will not be created if it does not exist, or if Python throws a File Not Found Error. If no mode is specified, this is the default mode.

The `'w'` mode is used to open a file for writing. The information in the file will be overwritten if the file already exists. If the file couldn't be found, then python automatically generates it.

When a file is opened in parallel with the append mode `'a'`, the file will be opened for write access, but existing data will remain unchanged. The data is appended at the end of the file instead of replacing existing data. If the file is not there, it is created, as it is in write mode.

Create mode `'x'`: New file for writing. If the file already exists it will fail with a `FileExistsError` and not alter anything, thus helping people avoid accidentally altering the file they mean to use for writing.

`'r+'`: The `"r+"` mode will open a file read-mode and also write-mode without truncating the file. The write-read mode `'w+'` creates a file for writing and reading but first overwrites the file.

Like any other language, Python also supports binary modes with the addition of the `'b'` to the mode string such as `'rb'` or `'wb'`. When operating on a file that's not a text file (such as an image, audio file, or executable) that needs to be treated as bytes, not text, you use binary mode.

This selection of the appropriate file mode is needed to avoid data loss, manage the file errors and interact with files as expected.

b) Methods of Adding New Data into a List

There are several inbuilt methods available in python that can be used to add elements to a list.

The `append()` function adds one element to the end of the list. It alters the actual list and doesn't return a fresh list. This is the standard technique used if adding an item one at a time.

`insert()` can be used to insert an element at an index in the list. It requires 2 arguments: index where the element is to be inserted, and the element to be inserted. It moves all the elements in that index on or after it to the right.

The `extend()` method is called to add several items to the end of a list. It can take any iterable (list, tuple or string) as its argument and appends each item of that iterable one by one to the initial list. `extend()`, as opposed to `append()`, unpacks and adds the items in the iterable.

Two lists can be combined and given a new list by using the concatenation operator (+). This does not make changes to the original list, it only makes a new list that contains elements from the original list.

Also, the repetition/reassignment scheme with `+=` can be used to extend a list in-place, adding elements from an iterable is analogous to `extend()`.

These two approaches have different uses depending on whether one wants to add a single element or many, or whether to add it at a particular spot.

Question 3

a) Difference Between Tuples and Lists, and Tuple Methods

The difference between tuples and lists in Python is they are both ordered sequences, but have a few key differences.

Mutability is the most basic of differences. Elements of a list can be changed after creation, and it is possible to add and remove elements. The elements of a tuple are fixed in place when it is created, once they exist they cannot be modified in any way.

The immutability of tuples makes them faster to iterate and access, than lists. They save less memory, and use it more efficiently if the data is not about to change.

List elements are enclosed in square brackets, tuples in parentheses. This syntactic distinction implies their purpose: lists are for artfully changing data, tuples are for permanently static data.

Since tuples are immutable and hashable, they can serve as keys for dictionaries. This is why lists can't be used to store keys for the dictionary.

There are many ways to modify a list, for example, `append()`, `remove()`, or `sort()`. Because Tuples are immutable, they don't have many built-in methods.

There are two inbuilt methods that could be used with tuples, `index()`, `count()`. The `count()` method is used to get the count of the value in the tuple. The `index()` method

return the position of the value that was found in the tuple. If not found, it throws ValueError.

b) Dictionary: Adding and Removing Data

Dictionary in Python: Unordered, Mutable Collection of key-value pairs. A dictionary key can be of any type, but each must be unique and must be unchangeable after the dictionary is constructed. The definition of a dictionary is done with curly braces, keys followed by values on a colon.

Data can be placed in a dictionary by just assigning it to a key with the assignment operator. If the key is not found, Python adds a new key value pair. If the key is found, its value is replaced by the new value. You can provide a dictionary and/or iterable of key-value pairs to update one or more key-value pairs using the update() method.

Python provides multiple methods to delete the elements from a dictionary. Key in the argument is the key of the element to be removed and the result is the value of the removed element. The pop() method removes the element with the key element and returns its value. It will throw a KeyError if the key is absent, but with a default if one is provided as the second argument. The popitem() function returns and deletes the last inserted pair from the dictionary. You can use the del statement to remove an individual key-value pair, or an entire dictionary. The clear() method empties out the dictionary by removing all the elements.

SET – II

Question 4

a) Variable Length and Keyword Arguments in Python

If you do not know how many arguments a function might need, you can write a function that takes an unspecified number of arguments. The ability to have an unspecified number of arguments is useful for writing functions when you don't know the exact number of arguments that the function needs.

The *args contains variable length arguments in the call to a function. If a single asterisk is used, it will capture the remaining optional positional arguments of the function into a tuple. If a single asterisk is used, then the remaining optional positional arguments of the function will be collected into a tuple. This enables to pass any number of positional arguments to a function other than the parameter types that have been specified. Within the function, arguments collected are accessible as usual, as a tuple.

The **kwargs syntax is used to deal with variable length argument lists in keywords, i.e., variable arguments that are preceded by a colon. If one has two stars prefixing it, it gathers all the remaining keyword arguments in dict. When the caller passes keywords, they're added to this dictionary as keys and values. This is helpful if the function has to accept names of parameters which are not known upon definition, and whose number varies.

You can have both `*args` and `**kwargs` in the same function. The standard position is for regular parameters first, then the tuple of non-keyword values, and last, the tuple containing the keyword-only parameters. The typical pattern is: regular parameters, then the tuple of non-keyword parameters, then the tuple of keyword-only parameters, and lastly the tuple of kwargs. This is strictly adhered to in Python.

These mechanisms result in Python functions being very flexible and reusable, so that general-purpose functions can take a wide variety of inputs without having to overload functions as is done in other languages.

b) Types of Recursion in Python

Recursion is a programming approach that involves using a function that calls itself on some smaller version of the problem. Strictly speaking, for each recursive call it must be possible to reach a "base case which terminates that recursive calls".

Direct recursion is a type of recursion wherein the function recursively calls itself directly. There are many examples involving activities such as computing $n!$ or computing Fibonacci numbers, the simplest examples of which involve this type of recursion.

Indirect recursion is the calling of one function from another function which calls the first function, which calls the second function, and so on... This is more complicated and not often used, but OK for some parsing and state-machine situations.

Tail recursion –

Last operation a function does is recursive call before returning. When working with tail recursive functions, it's not necessary to store the previous state of the function after a call. In general, some languages are good at optimizing tail recursion in order to prevent stack overflow when it gets too deep in the tail; although this optimization is not performed by default in Python.

Head recursion: Recursive call is made as the first operation in the function. All recursive calls return and the result of one recursive call is used by the current call. All recursive calls have returned the result, which is then passed back to the current call.

Tree recursion: A function can make multiple recursive calls inside of one call, creating a branching call tree. It is a common example of a recursive program to compute Fibonacci numbers as a naive recursive program, which is a classic example of tree recursion.

Linear Recursion-

When a function calls itself only once per call. Only processes one item at a time moving toward the base case, step by step.

Question 5

Encapsulation and Class Variables vs Instance Variables

Object-

oriented programming basics include encapsulation. It is a design principle that involves relating data (attributes) and operations on these data in one unit (class). The main thing of encapsulation is to prevent direct access to some parts of an object and to prevent changes to the internal state by the surrounding code.

Encapsulation in python is done by the use of access modifiers. If the name of an attribute is preceded by one underscore, this signifies that it is for use internally and cannot be accessed from outside of the class (unlike norm in Python). The double underscore prefix causes name mangling which results in a greater degree of protection since the attribute is less accessible from outside the class. Encapsulation enables the programmer to provide a specified interface to the outside world without revealing how it works, thus improving the ease of program maintenance, complexity and security of the program.

Encapsulation also supports the principle of data hiding: in this case the internal representation of an object is not exposed, and access to it is made available only via clearly-defined methods, known as getters and setters. Consistently ensuring that any validation or processing logic that is supposed to be applied to data will be doing it every time.

The difference between Class variables and Instance variables is which one is more widely used and how. A variable declared within the class but not within any function/method is a class variable. This is shared amongst all the instances of the class – that is if one object changes a class variable, then the changes are visible to all other objects that did not override the class variable. Class variables can be useful to hold data shared by all instances, like a counter to keep track of the number of objects created.

An instance variable is defined within a method (usually the constructor method `init`) and refers to it with `self`. There is a separate copy of instance variables for each object. The values of the instance variables of an object are independent of other objects. Instance variables hold information specific to an instance of a class, like a person's name and age.

Class variables are variables that have a shared state and instance variables are variables that have an individual state. In conclusion, class variables have a shared state, whereas instance variables have an individual state and each serve a unique purpose in the object-oriented design.

Question 6

Database Connectivity in Python Programming and DDL/DML commands.

There are inbuilt and third party library for connecting as well as interacting with databases in python. The most popular library tools for connecting to MySQL are `mysql-connector-python`; and SQLite can be connected with using the built-

in sqlite3 module. Whatever type of database system you use, the steps for connecting to it have a common thread.

First step is to import necessary database module. In case of MySQL, it is `mysql.connector`, and for SQLite it is `sqlite3`.

The second part is to establish a link to the database by using the `connect()` function. Parameters include host, user, password and database. The `connect()` function returns a handle to a connection with the database.

The third is to use the `cursor()` method on the connection object to create a cursor object. The cursor is used to point or handle to point to or handle a SQL statement and get results back from the database.

Once you have fulfilled the previous steps, you can perform SQL operations on the cursor, through the `execute()` method. The SQL command is passed in as a string argument. With Data Definition Language commands, the execution changes the structure of the database. Execution changes data in tables in the case of Data Manipulation Language commands.

The fifth step is to commit the transaction with the use of the `commit()` method of the connection. For DML operations to save the changes permanently to the database, this is necessary. Some databases automatically commit any changes made to the data.

The sixth step is to fetch results when a SELECT query has been executed. The returned rows are retrieved using some of these methods: `fetchone()`, `fetchall()`, or `fetchmany()`.

The last one is to close the Cursor and the connection, releasing resources, using the `close()` method on each of them.

Data Definition Language (DDL): These are the commands that are used to define and alter the structure of a database. The CREATE command creates new tables, databases or other objects. ALTER –

changes the structure of a table, which can include adding or dropping a column. Remove a table or database by using the DROP command. These commands work with the schema and not the data.

DML is the Data Manipulation Language which is a set of commands that work directly with the data in the tables. The INSERT statements create new rows to a table. The UPDATE command updates existing records (makes changes) when a condition is met. The DELETE statement to delete records from a table on the basis of a condition. **SELECT:** Used to fetch data, sometimes considered part of DQL, but in practice its usage is closely related to DML.

DDL and DML constitute the fundamental operations of SQL where DDL are used for managing the structure and DML are used for managing the content of the database.